LLM ENGINEERING

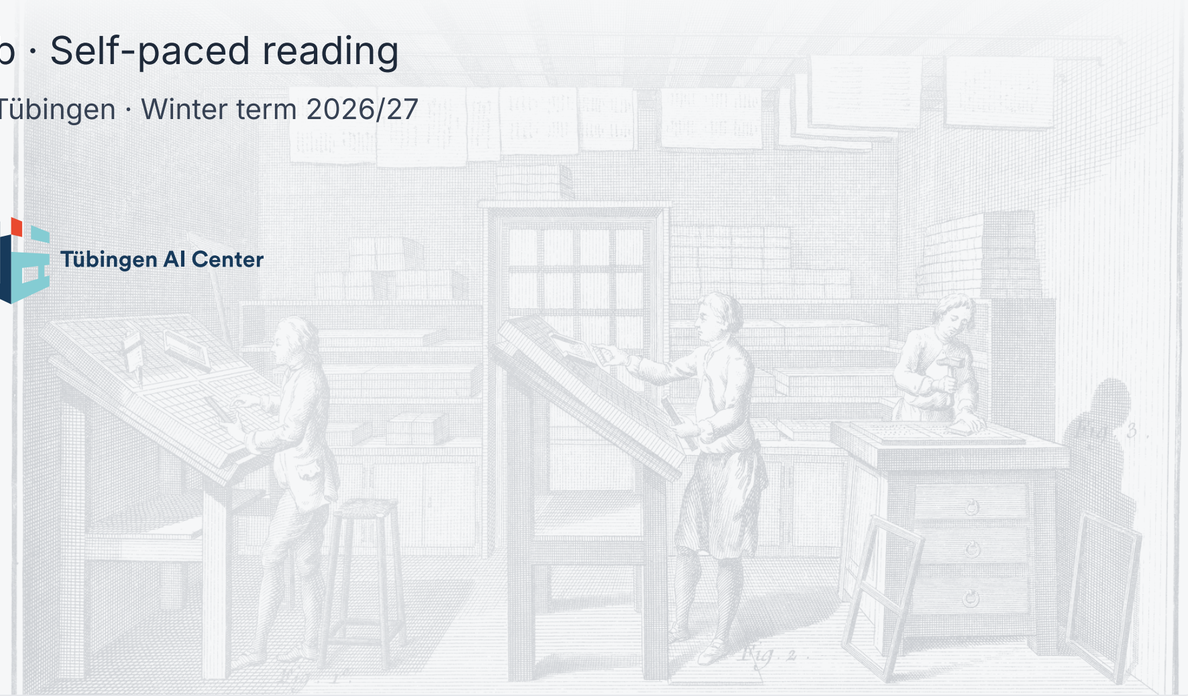
Bonus chapter 1

Bonus: Tokenization

Bethgelab · Self-paced reading

University of Tübingen · Winter term 2026/27

Tübingen AI Center



Why not feed the model raw bytes?

Every text is already a sequence of bytes.

Words, bytes and subwords trade vocabulary for length

"The transformer architecture revolutionized natural language processing."

Unit	Vocabulary	Unknown inputs	Positions
Words	One row per word type	Unseen words	8
Bytes	256	None	72
Monad subwords	8,192	A few characters	10

- Bytes: one position per byte
- Subwords: fixed vocabulary, shorter sequences

Training learns ranks; encoding replays them

- Tied counts: the earliest pair in the corpus wins
- Encoding applies the ranks in order

Toy character-symbol BPE training on one phrase

Corpus: “the lower the better”

Initial alphabet: ␣ b e h l o r t w

Tie rule: earliest pair in the current corpus wins.

Stop: when no adjacent pair occurs more than once.

Training step 0 of 4 · 20 current symbols

Reset | Previous merge | Apply merge

t h e ␣ l o w e r ␣ t h e ␣ b e t t e r

Selected pair: t + h (frequency 2) → th

Learned merge ranks: none yet

BPE learns an ordered list of merges

- **Byte Pair Encoding** (BPE) (Sennrich et al., 2016)
- Start from base symbols, such as characters or bytes
- Merge the most frequent adjacent pair
- Record each merge's rank
- Stop at a vocabulary-size or pair-frequency target

How does BPE encode “other”?

Use the same four learned merges.

Encoding applies ranks in order

- `[o, the, r]`: 5 symbols, 3 tokens
- Rank 2, `th + e`, consumes the `e`
- Rank 4, `e + r`, then never applies

A 256-byte base leaves no input unknown

- Base: all 256 byte values (Radford et al., 2019)
- `Ġ` displays the space byte
- UTF-8: 1 to 4 bytes per character (Yergeau, 2003)
- Composed (NFC) `é` is bytes `C3 A9`
- Monad's vocabulary keeps 207 of 256 byte symbols
- SentencePiece: another design, trained on raw sentences (Kudo & Richardson, 2018)

Pre-tokenization fixes where merges may happen

- A regular expression cuts text into chunks (OpenAI, 2019)
- Merges never cross a chunk
- Llama 3: chunks of one to three digits (Meta, 2024)
- Monad keeps each digit run whole (PleIAs, 2025)

```
"Pay 1000000 now"
chunks: ["Pay", "Ġ1000000", "Ġnow"]
tokens: ["P", "ay", "Ġ100", "0000", "Ġnow"]
```

Byte coverage does not guarantee a round trip

- Composed (NFC) `é`: `C3 A9`
- Decomposed (NFD) `é`: `65 CC 81`
- Normalization can change the code points
- Record every preprocessing policy

Special tokens mark structure, not text

```
id 0  [UNK]               no symbol for a byte
id 1  <|begin_of_text|>   start of a sequence (BOS)
id 2  <|end_of_text|>     end of a sequence (EOS)
id 3  [PAD]               filler for short rows
```

- Beginning and end of sequence, BOS and EOS
- The course adds EOS after every document
- Padding fills short rows, and masks hide it
- Literal `<|end_of_text|>` in text becomes ID 2
- Week 11's chat template ends each turn with EOS

Every vocabulary row costs d_model parameters per table

$$\text{parameters per table} = V \cdot D$$

- Vocabulary size V, model width D (`d_model`)
- Input embedding: one table
- Untied output head: a second table
- Course: $8{,}192 \times 512 = 4{,}194{,}304$ parameters

How many parameters are added?

The vocabulary grows from 8,192 to 32,768 entries at d_model = 512.

Quadrupling V adds 12.6M parameters per table

- 24,576 new rows of 512 parameters each
- Tied head: 12,582,912 more parameters
- Untied head: 25,165,824 more, since both tables grow

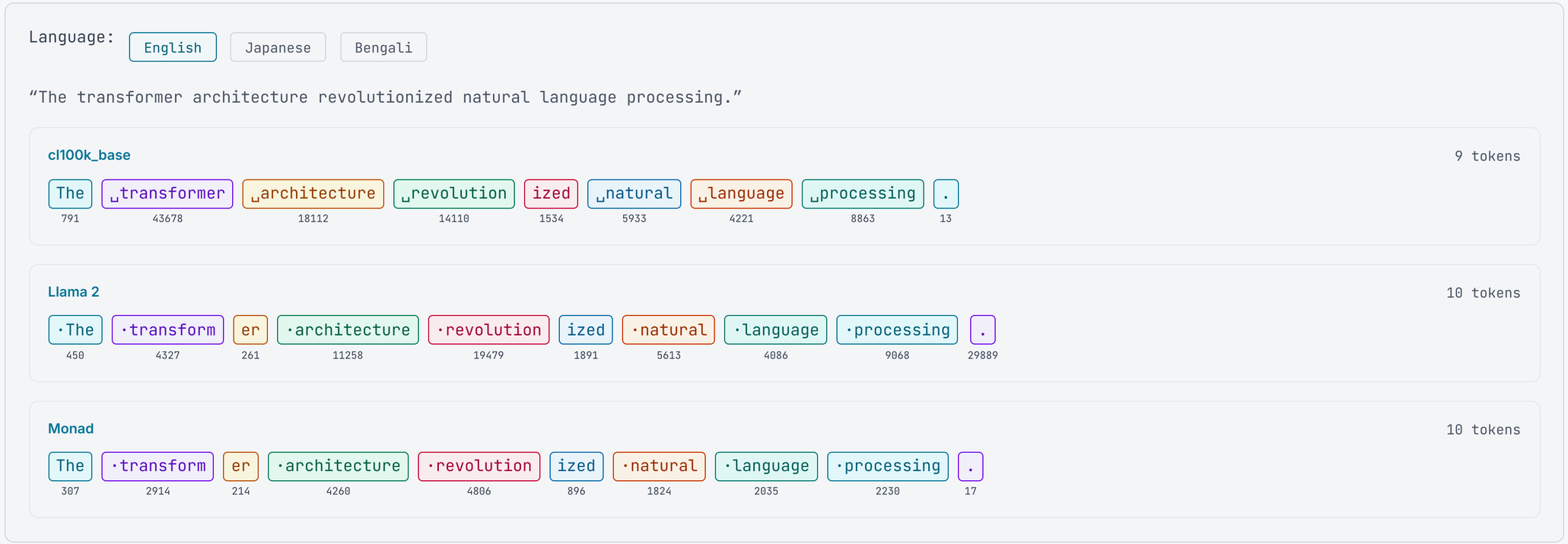
More rows shorten sequences but thin the training signal

Dimension	Effect	Measure
Model tables	$V \cdot D$ per table	Parameters and memory
Sequence length	Larger V may shorten a corpus	Token count, effective context
Piece frequency	Signal spread over more pieces	Frequency and fragmentation
Decision	No size best everywhere	Target-language and domain samples

- Compare candidates on the target data mixture (Penedo et al., 2025)
- Opus 4.7's new tokenizer: 1.0 to 1.35 times as many tokens (Anthropic, 2026)

Compare tokenizers on one text, not languages on different texts

- `cl100k_base`: OpenAI's GPT-4 tokenizer
- Monad needs up to 2.5 times `cl100k_base`'s tokens
- Three sentences, not translations
- Monad: Japanese and Bengali stay one token per byte



Three tokenizers split 342792 three ways

- Arithmetic reuses place value and carries
- Merges follow frequency, not arithmetic
- Irregular pieces invite plausible, wrong answers
- `_` marks a word boundary

cl100k_base

[342] [792]

Llama 2

[_] [3] [4] [2] [7] [9] [2]

Monad

[34] [27] [92]

Where does Monad cut 1,342,792.05?

Its pre-tokenizer keeps digit runs whole.
Can the chunks alone tell you
whether 342 stays one token?

Monad splits 342 differently in each context

```
"1,342,792.05"
chunks: ["1", ",", "342", ",", "792", ".", "05"]
tokens: ["1", ",", "34", "2", ",", "7", "92", ".", "05"]
```

- Chunks break at every comma and the decimal point
- Chunks alone cannot tell, so measure the merges
- Between commas: `34 · 2`
- In `342792`: `34 · 27 …`
- After a space: `Ġ3 · 42`

Boundaries change the arithmetic a model must learn

- Digit alignment and grouping direction changed arithmetic accuracy (Singh & Strouse, 2024)
- Test the number formats your system serves
- Include signs, decimals, separators and all served lengths

Fertility and PCW count how often words split

- Fertility: tokens per segmented word
- Proportion of continued words (PCW): share of words split (Rust et al., 2021)
- Name the corpus and word segmenter
- No universal pass or fail thresholds

What are the fertility and PCW?

Monad encodes “tokenization matters”
as to · ken · ization · Ġmatters.

Fertility 2 and PCW 0.5 for one short phrase

- 4 tokens, 2 words
- Fertility: $4/2 = 2$
- PCW: 1 of 2 words split, so 0.5

Each denominator answers a different question

Japanese sentence, `cl100k_base`	Value
Tokens	26
UTF-8 bytes	66
Code points	22
Tokens per byte	0.394
Tokens per code point	1.182

- In ASCII, bytes and code points coincide
- Word segmentation depends on the language

Reuse a tokenizer unless measurements justify training one

- Ask four questions: data, model, system, outcome
- Training one costs compatibility and validation (Dagan et al., 2024)
- Libraries: `AutoTokenizer`, tiktoken, Gigatoken

Situation	Starting point	Entries
Very small model, English only	Monad	8,192
Medium-sized model	Llama 2	32,000
Medium-sized model	`cl100k_base`	~100k
Many languages, larger tables	`o200k_base`	~200k
Many languages, larger tables	Gemma 4	~262k

So why not feed the model raw bytes?

Raw bytes cost length; subwords buy it back

- Course text: 4.17 bytes per token
- Merges spend vocabulary rows to shorten sequences
- Full coverage needs all 256 bytes
- Monad keeps 207 byte symbols, enough for course text
- Course tokenizer: Monad, 8,192 entries (PleIAs, 2025)

Six results on tokenization

1. Subwords trade vocabulary size for length
2. BPE learns ranks, and encoding replays them
3. Check byte coverage, chunks and normalization
4. Vocabulary rows cost parameters and shorten sequences
5. Number boundaries change the arithmetic a model must learn
6. Measure on target data before choosing

Reflection questions

No submission. Bring short answers to the Week 2 tutorial.

1. Monad's one-to-three-digit rule is stored as a literal string, so it never runs. Suppose you turn it into a working regular expression but keep Monad's merges and the model trained with them. Does Week 1's sentence "The price is 342 euros." still decode to the same text, how do its tokens change, and what does the model now see that it never saw in training?
2. You are building the tokenizer for a small model that reads English and German product reviews, with prices such as "1.299,00 €" and words such as "Größe". Choose a byte policy, a rule for digits and a vocabulary size, and name the measurements, on which text, that would make you change each choice.
3. Claude Opus 4.7's new tokenizer maps the same input to up to 1.35 times as many tokens. Using what this chapter shows about vocabulary rows, piece frequency and number boundaries, give two reasons a lab might accept more tokens per text, and say what you would measure before making such a switch yourself.